

The Normal Equations Are Normally Faster: Benchmarking Ordinary Least Squares Solvers for $n \geq p$

Albert Dorador
trustalgorithm.dev@gmail.com

November 26, 2025

Abstract

For over five decades, numerical analysis textbooks have warned against forming the normal equations $X^T X \beta = X^T y$ for ordinary least squares on numerical stability grounds. We show that in the ubiquitous tabular regime ($n \geq p$, dense double-precision data without perfect multicollinearity), directly solving the unregularized normal equations via LU or Cholesky decomposition is not only numerically safe but substantially faster than the often-recommended SVD or QR-based approaches. Across 12 representative scenarios with $n \in [20, 50000]$ and $p \in [2, 1000]$ including $n = p = 50$ and $n = p = 1000$, we consistently measure speedups of $3\text{--}18\times$ using 1000 individually timed per-solve repetitions, with negligible coefficient disagreement ($\lesssim 10^{-9}$). Despite the algorithmic superiority of Cholesky-based approaches, solving the normal equations by LU decomposition exhibits comparable performance due to the use of highly optimized LAPACK routines. For this reason, the simplicity of the LU-based approach makes it a compelling choice in high-throughput tabular pipelines requiring repeated least squares solves, delivering improved performance without paying the numerical stability price that is often assumed.

1 Introduction

Forming the normal equations $X^T X \beta = X^T y$ has been considered poor practice since the 1970s [5] due to potential loss of accuracy from squaring the condition number of the design matrix [4]. Standard advice favors QR decomposition or SVD-based solvers such as `scipy.linalg.lstsq`.

However, in many domains of tabular machine learning, problems are routinely well-behaved enough for modern solvers to handle with satisfactory precision. This is especially true in the common $n \geq p$ case without perfect multicollinearity. Moreover, adding ridge regularization ($\lambda > 0$) largely mitigates the challenges that the underdetermined case poses.

Under these conditions, $X^T X + \lambda I$ (with $\lambda \geq 0$) is symmetric positive-definite, which allows the for the normal equations to be solved via Cholesky decomposition, solving triangular systems of equations. This is the most theoretically efficient way to solve the normal equations, requiring only about $p^3/3$ flops for a $p \times p$ Gram matrix, while e.g. using LU decomposition requires twice the work ($2p^3/3$ flops) [4]. Being able to rely on such a fast algorithm can be crucial in applications that require solving many independent least-squares problems, such as:

- linear model tree fitting, for a tree of depth d , n samples and p features, typically in the order of $\mathcal{O}(2^d \times n \times p)$ with naive routines; even with state-of-the-art routines, the number of independent, full ordinary least squares (OLS) solves required is at least $\mathcal{O}(2^d)$
- calculation of variance inflation factors (VIF) or ghost variables [2, 3], in the order of $\mathcal{O}(p_n)$, where p_n is the number of numeric variables in your design matrix X

This work presents simple, reproducible benchmarks showing that direct solution of the unregularized normal equations is nowadays numerically safe even in square designs with large condition numbers, while always being the fastest OLS approach by a large margin.

2 Methods

We compare five standard approaches on synthetic dense data without an intercept term:

1. SVD-based (`scipy.linalg.lstsq`)
2. Manual QR decomposition + LU solver (`np.linalg.solve`)
3. Normal equations with Gram inverse (`np.linalg.inv`)
4. Normal equations with LU solver (`np.linalg.solve`)
5. Normal equations with Cholesky solver (`scipy.linalg.cholesky`)

Design matrices X and response vectors y were generated as i.i.d. standard Gaussian entries, which is common practice in the literature and avoids arbitrary choices of correlation structures and signal-to-noise ratios (the latter having no impact in solving times or stability).

1000 individually timed per-solve repetitions were carried out per method and scenario, allowing us to calculate error bands around the mean recorded times. We do not assume normality of the recorded times and instead consider two standard errors (which, in the absence of normality, may not correspond to a traditional 95% confidence interval but in practice it is likely in its vicinity). These error bands provide a sense of ‘significance’ that can be valuable when analyzing the results.

Both absolute times as well as relative times to SciPy’s `lstsq` are reported. The choice of baseline obeys to the popularity of the `scipy.linalg.lstsq` routine to solve OLS problems, being the method of choice of the mainstream `scikit-learn` machine learning library, explicitly stated for instance in `sklearn.linear_model.LinearRegression`.

A seed (123) was fixed to guarantee partial reproducibility (e.g. condition numbers). Exact, full reproducibility is impossible to guarantee as runtimes (especially at microsecond scale) are sensitive to factors that can’t be easily controlled such as concurrent computer processes. The code to obtain our results is accessible at <https://github.com/adc-trust-ai/trust-free>.

Experiments were run on Python 3.11.13, NumPy 1.26.4, SciPy 1.16.3, on an Apple M4.

3 Results

Wall-clock times for 1000 repetitions per method and scenario are reported in Table 1. Figure 1 visualizes speedups relative to SciPy’s `lstsq`.

Table 1: Mean time (ms) $\pm$ 2 std. errors across 1000 repetitions. Results in bold show the fastest method(s) up to error-band precision. κ denotes condition number of design matrix.

n	p	κ	N.E. + Chol	N.E. + LU	N.E. + inv	QR + LU	SciPy’s <code>lstsq</code>
20	2	1.1	0.011 $\pm$ 0.001	0.003 $\pm$ 0.000	0.004 $\pm$ 0.000	0.010 $\pm$ 0.001	0.013 $\pm$ 0.001
20	15	6.2	0.018 $\pm$ 0.001	0.053 $\pm$ 0.016	0.067 $\pm$ 0.043	0.053 $\pm$ 0.016	0.052 $\pm$ 0.003
50	50	192	0.035 $\pm$ 0.002	0.048 $\pm$ 0.004	0.066 $\pm$ 0.005	0.134 $\pm$ 0.016	0.303 $\pm$ 0.008
500	5	1.1	0.024 $\pm$ 0.002	0.011 $\pm$ 0.000	0.023 $\pm$ 0.004	0.036 $\pm$ 0.002	0.044 $\pm$ 0.001
500	20	1.4	0.035 $\pm$ 0.001	0.039 $\pm$ 0.004	0.066 $\pm$ 0.011	0.200 $\pm$ 0.015	0.178 $\pm$ 0.008
1K	1K	3.5K	0.012 $\pm$ 0.000	0.013 $\pm$ 0.000	0.035 $\pm$ 0.000	0.047 $\pm$ 0.000	0.087 $\pm$ 0.001
5K	5	1.1	0.072 $\pm$ 0.001	0.062 $\pm$ 0.002	0.064 $\pm$ 0.002	0.212 $\pm$ 0.005	0.195 $\pm$ 0.004
5K	100	1.3	0.001 $\pm$ 0.000	0.001 $\pm$ 0.000	0.002 $\pm$ 0.000	0.011 $\pm$ 0.000	0.012 $\pm$ 0.000
5K	500	1.9	0.010 $\pm$ 0.000	0.010 $\pm$ 0.000	0.026 $\pm$ 0.000	0.091 $\pm$ 0.001	0.092 $\pm$ 0.001
50K	5	1.0	0.001 $\pm$ 0.000	0.001 $\pm$ 0.000	0.001 $\pm$ 0.000	0.003 $\pm$ 0.000	0.002 $\pm$ 0.000
50K	100	1.1	0.008 $\pm$ 0.000	0.008 $\pm$ 0.000	0.015 $\pm$ 0.000	0.182 $\pm$ 0.001	0.145 $\pm$ 0.001
50K	500	1.2	0.112 $\pm$ 0.002	0.116 $\pm$ 0.003	0.244 $\pm$ 0.003	1.279 $\pm$ 0.028	0.681 $\pm$ 0.003

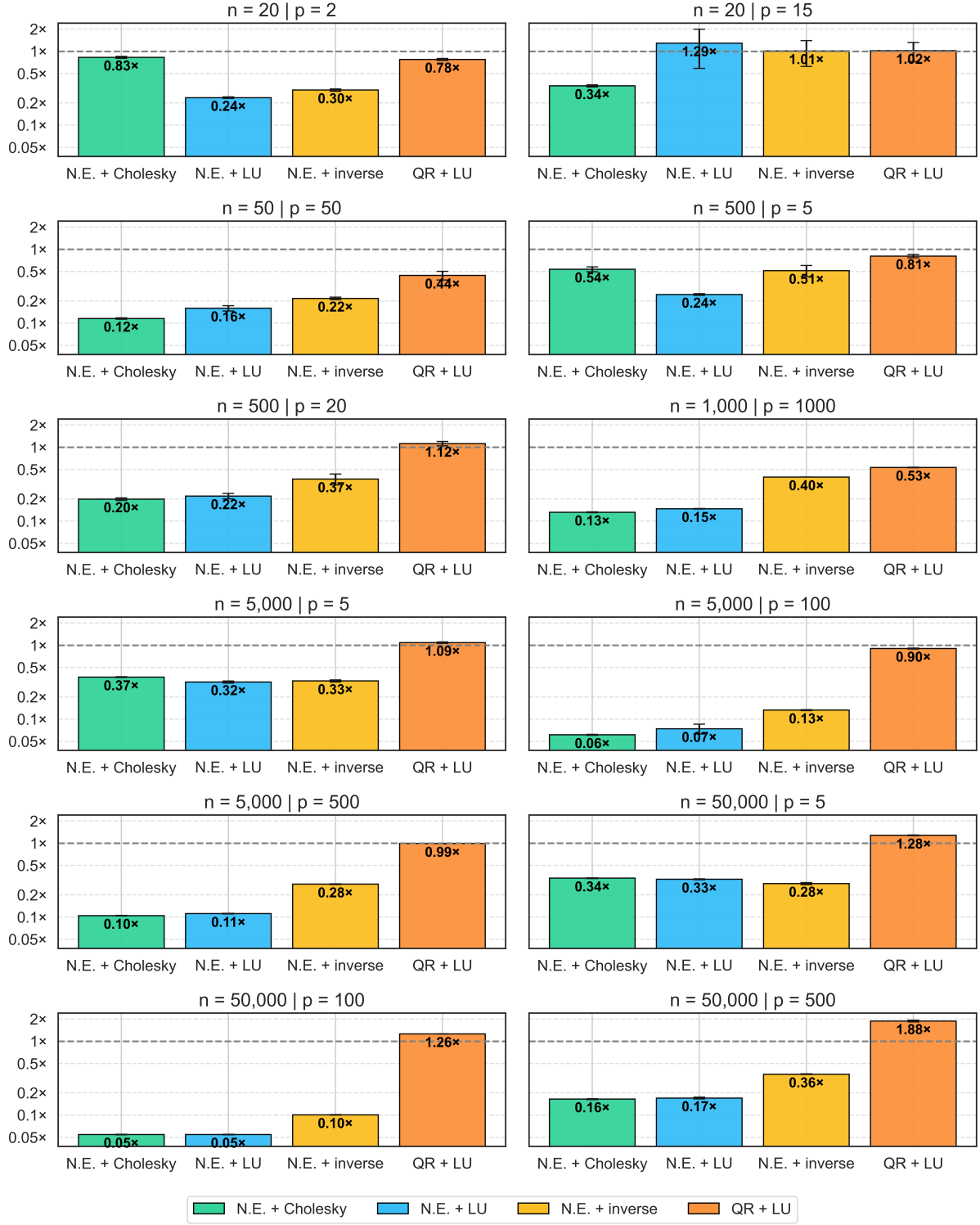


Figure 1: Runtime relative to SciPy's lstsq with $2\times\text{SE}$ error bands (logarithmic scale)

Maximum coefficient difference across all methods was $\lesssim 10^{-9}$, confirming numerical equivalence. Solving the normal equations via Cholesky decomposition appears slightly less numerically stable (for large condition numbers) than other methods to solve said equations, like a LU-based approach, which can be implemented in one user-friendly line of Python code.

4 Discussion and Recommendation

The results we obtained suggest that forming the normal equations is, by far, the fastest way of solving OLS problems in the overdetermined regime without perfect multicollinearity. In addition, coefficient estimates in all cases are within at most $\sim 10^{-9}$ of those estimated with the universally-recommended SVD or QR-based methods, dismissing long-ingrained numerical stability concerns. Note that these stability results include the two square design cases, which should be challenging for this approach.

Since the consensus seems to still favor working with the SVD or QR decompositions of the design matrix, it must be the case that at least one of the two results we showed (superior computational speed and nearly equivalent numerical stability) is largely unknown to the community, which makes this contribution valuable.

Nonetheless, these results should not come as a big surprise if we keep in mind two elements: first, the rule of thumb that says that if the condition number of matrix X is 10^k , then you may lose up to k digits of accuracy on top of what would be lost due to arithmetic machine precision when solving a linear system with that condition number [1]; second, the fact that modern hardware runs on 64-bit architectures, meaning that even with an extremely ill-conditioned design matrix having a condition number of 10^6 , and thus a corresponding Gram matrix with a condition number of 10^{12} there would still be at least 4 digits of accuracy left, which may be enough in many applications.

Indeed, our results reveal that inverting the Gram matrix is very safe numerically even under $n = p$, and tends to be substantially faster than SVD-based approaches (even for larger p). Still, solving the normal equations via LU decomposition (`numpy.linalg.solve`) appears to be uniformly superior and hence we won't emphasize the inversion method in the remainder of this paper.

Another fact worth mentioning is that the modern LAPACK `_gesv` routines to solve OLS that are called by `numpy.linalg.solve` are about as fast as direct Cholesky-based approaches, despite using LU decomposition which is about twice as compute expensive as the Cholesky decomposition (and uses twice as much memory, as it needs to store both L and U matrices). There might be several explanations for this phenomenon. First, it is possible that the common Gram matrix formation step $X.T @ X$ involving $\mathcal{O}(np^2)$ operations dominates runtime, so the difference in matrix factorization time may be relatively small. Second, LAPACK `_gesv` uses a highly optimized, likely multithreaded BLAS inside the LU factorization step, whereas the solver for triangular matrices employed in a direct Cholesky-based approach (`scipy.linalg.solve_triangular`) is not multithreaded because it is memory-bound and not compute-bound. Another reason is differences in overhead: the Cholesky-based approach requires a first call to SciPy's Cholesky factorizer, plus two calls to SciPy's triangular matrix solver, while the LU approach only requires one function call, allowing better caching and batching. These differences can be significant for small p : indeed, the Cholesky approach is never slower for $p > 5$.

Therefore, while using a Cholesky decomposition is the theoretically optimal way of solving the normal equations, for small p LU-based approaches might be faster in practice, and even up to moderate values of p (say, 1000), the difference in speed is often negligible. Therefore, in practice solving the normal equations via LU decomposition (`np.linalg.solve`) remains an attractive choice due to its comparable performance and simpler setup, requiring just one line of code: `numpy.linalg.solve(X.T @ X, X.T @ y)`. Additionally, if NumPy is already being used (very likely) this approach avoids relying on one more dependency (SciPy). Moreover, the Cholesky-based approach appears slightly less stable numerically, especially with design matrices that are less well-behaved (since Cholesky, unlike LU, requires positive definiteness).

For these reasons, we recommend using the one-liner `numpy.linalg.solve(X.T @ X, X.T @ y)` as the default OLS solver in overdetermined problems with moderate size (below 1000

variables), often resulting in 3–18× speedups over the universally-recommended QR or SVD-based solvers (e.g. `scipy.linalg.lstsq`). For larger, well-behaved problems, the Cholesky-based solution might be even faster while remaining stable, and thus preferable. In any case, the speedup that directly solving normal equations provides is bound to make a material difference in applications requiring repeated OLS solves.

Lastly, note that the recommended approach can be easily adapted to solve underdetermined systems (e.g. in the $n < p$ regime, common in domains like genomics) by adding a ridge penalty, and solving instead the L_2 -regularized normal equations

$$(X^T X + \lambda I)\beta = X^T y \quad (1)$$

where I denotes a $p \times p$ identity matrix. Importantly, the solution β^* can be made arbitrarily close to the minimum-norm solution that `scipy.linalg.lstsq` provides by choosing an arbitrarily small value of $\lambda > 0$.

In general, adding a small ridge penalty $\lambda > 0$ could be desirable for numerical stability as an essentially free ‘insurance policy’ against less well-behaved design matrices that can be encountered in practice. If λ is made to decay to 0 at a rate no slower than $1/\sqrt{n}$, then the coefficient estimates would be asymptotically unbiased, and, in general, all of the asymptotic OLS guarantees of the original, unregularized approach would be preserved.

However, for $n \ll p$, forming the (regularized) normal equations is not advised for a simple reason: the dimensions of the Gram matrix ($p \times p$). Working with such a large matrix would lead to slower solutions compared to e.g. SVD-based approaches. Not only that, storing in memory a $p \times p$ matrix (p^2 entries) can easily make the machine run out of memory, making this method unusable in most machines for $p \gtrsim 10^5$.

5 Conclusion

For over 50 years, textbooks have warned against forming the normal equations to solve OLS problems due to ill-conditioning concerns. Our results suggest that these concerns are largely obsolete in the common tabular regime $n \geq p$: the direct solution via LU decomposition (which can be implemented in one line of Python code) is not only numerically safe but 3–18× faster than the often-recommended SVD or QR-based approaches. Solving the normal equations via Cholesky decomposition in the (common) positive-definite case is even faster in high dimensions, but for moderate-size problems the difference is often negligible. Solving the normal equations in underdetermined regimes (e.g. perfect multicollinearity or $n < p$) can be accomplished by simply introducing a small ridge penalty, which ensures positive-definiteness. However, for $n \ll p$, forming the normal equations is not advised due to the dimensions of the Gram matrix.

References

- [1] Cheney, W. and Kincaid, D. (2008). *Numerical Mathematics and Computing* (6th ed.). Thomson Higher Education, Belmont.
- [2] Delicado, P. and Pena, D. (2023). *Understanding complex predictive models with ghost variables*, TEST 32: 107–145.
- [3] Dorador, A. (2025). *TRUST: Transparent, Robust and Ultra-Sparse Trees*, PRICAI 2025. Lecture Notes in Artificial Intelligence, Springer Nature (forthcoming). arXiv: 2506.15791.
- [4] Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- [5] Lawson, C.L. and Hanson, R.J. (1974). *Solving Least Squares Problems*. Prentice-Hall.